```
# Inputs: ONLY outputs of script 26 (24 roots + 3 weight sets of 8, all derived from
T) plus the
# explicitly-stated construction rule below. No E8 data used.
import numpy as np, io, contextlib, itertools
with contextlib.redirect_stdout(io.StringIO()):
exec(open('26_roots_from_form_only.py').read())
Ch=np.linalg.cholesky(sc*Kinv)
roots=R@Ch                                  # 24 x 4, |x|^2 = 2
W={k:v@Ch for k,v in Wt.items()}            # 8 x 4 each, |x|^2 = 1
Wl=list(W.values())
print("roots norm^2 set",sorted(set(np.round((roots**2).sum(1),6))),"| weight
norm^2",[sorted(set(np.round((w**2).sum(1),6))) for w in Wl])
Z=np.zeros((1,4))
def build(M):   # RULE: (r,0), (0,M r) for the 24 roots; (w, M w') for w,w' in the
SAME weight set
    S=[np.hstack([roots,np.zeros((24,4))]), np.hstack([np.zeros((24,4)),roots@M.T])]
    for w in Wl:
        a=np.repeat(w,8,axis=0); b=np.tile(w@M.T,(8,1)); S.append(np.hstack([a,b]))
    return np.vstack(S)
def cheap(S):
    G=S@S.T; ok=np.abs(G-np.round(G)).max()<1e-8 and np.abs(G).max()<2+1e-8
    return ok,G
def full(S):
    key=lambda v:tuple(np.round(v,5)+0.0)
    st={key(v) for v in S}
    if len(st)!=240: return False
    for a in S:
        Sr=S-np.outer(S@a,a)
        if any(key(v) not in st for v in Sr): return False
    return True
S0=build(np.eye(4)); ok,G=cheap(S0)
print("\n[M = identity]  count:",len(S0)," unique:",len({tuple(np.round(v,5)+0.0) for
v in S0})," integer products in [-2,2]:",ok)
print("   reflection-closed (a root system):",full(S0) if ok else "skipped (cheap
test failed)")
# enumerate Aut(root set of D4): orthogonal M with M(roots)=roots
key=lambda v:tuple(np.round(v,5)+0.0); rs={key(v):i for i,v in enumerate(roots)}
# basis of 4 independent roots
B=[roots[0]]
for r in roots:
    if np.linalg.matrix_rank(np.array(B+[r]),tol=1e-8)>len(B): B.append(r)
    if len(B)==4: break
B=np.array(B); Gb=B@B.T; A=[]
def rec(img):
    k=len(img)
    if k==4:
```

```
        M=np.array(img).T@np.linalg.inv(B.T)   # M @ B^T = img^T
        if all(key(M@r) in rs for r in roots): A.append(M)
        return
    for r in roots:
        if all(abs(r@img[j]-Gb[k,j])<1e-8 for j in range(k)) and abs(r@r-Gb[k,k])<1e-
8: rec(img+[r])
rec([]); print("\nautomorphisms of the 24-root set found:",len(A))
res=[]
for M in A:
    S=build(M); ok,_=cheap(S)
    if ok and full(S): res.append(M)
print("alignment choices M giving a closed 240-element root
system:",len(res),"of",len(A))
print("   identity among them:",any(np.allclose(M,np.eye(4),atol=1e-8) for M in res))
# invariants of the first successful construction
S=build(res[0]) if res else S0; G=np.round(S@S.T).astype(int)
cnt={v:int((G[0]==v).sum()) for v in (-2,-1,0,1,2)}; print("\nrelation-value counts
for element 0:",cnt)
print("all 240 elements have identical counts:",all({v:int((G[i]==v).sum()) for v in
(-2,-1,0,1,2)}==cnt for i in range(240)))
ev=np.linalg.eigvalsh(S@S.T); print("gram eigenvalues: nonzero count",int((ev>1e-
8).sum()),"values",sorted(set(np.round(ev[ev>1e-8],6))),"|
rank",np.linalg.matrix_rank(S,tol=1e-8))
# simple roots -> Dynkin diagram
g=np.random.RandomState(1).randn(8); pos=[i for i in range(240) if S[i]@g>0]; ps=
{key(S[i]):i for i in pos}
simple=[i for i in pos if not any(key(S[i]-S[j]) in ps for j in pos)]
C=np.round(S[simple]@S[simple].T).astype(int); print("positive roots:",len(pos),"
simple roots:",len(simple)," det(Cartan) =",round(np.linalg.det(C)))
adj=(C==-1).astype(int); deg=adj.sum(1); print("edges:",adj.sum()//2,"
degrees:",sorted(deg))
br=int(np.argmax(deg)); arms=[]
for nb in np.where(adj[br])[0]:
    prev,cur,L=br,nb,1
    while adj[cur].sum()==2:
        nxt=[x for x in np.where(adj[cur])[0] if x!=prev][0]; prev,cur=cur,nxt; L+=1
    arms.append(L)
print("branch-node arm lengths:",sorted(arms),"(E8 diagram has arms 1,2,4)")
```